

Health & Fitness Text Analytics

Word2Vec embeddings and a complete text preprocessing pipeline

Project report — how the project was built, the datasets used, how the text was cleaned, the results, and the answers to every interpretation question.

Name	_____
Roll number	_____
Tools	Python 3.12, NLTK, Gensim, pandas, Streamlit, Plotly
Repository	github.com/SayAn1-dls/ml_bba

Contents

1. Problem statement
2. Datasets
3. Tools and libraries
4. Q1 — Word2Vec: method
5. Q1 — Results
6. Q1 — Interpretation
7. Q2 — Preprocessing: how the text was cleaned
8. Q2 — Tokenization, stopwords, stemming, lemmatization
9. Q2 — Comparison and vocabulary
10. Q2 — Interpretation
11. Interactive web app
12. Project structure and how to run
13. Quick revision: likely viva questions

1. Problem statement

Computers cannot read text the way we do — they work with numbers. Before any NLP model can use text, the text has to be **cleaned** (remove noise), **split into words** (tokenization), **reduced** to useful words (stopword removal, stemming, lemmatization) and finally **turned into numbers** (for example word embeddings). This project covers both halves of that process using small health & fitness text.

Question 1 — Word2Vec

Train a Word2Vec model with Gensim on 20 given sentences, print the vocabulary, the vector for the word **diet**, the top-5 most similar words for **diet** and **workout**, and explain whether the results make sense and why a small dataset gives noisy results.

Question 2 — Text preprocessing pipeline

Take 12 raw, noisy sentences (mixed case, punctuation, numbers, emoticons, extra spaces), clean them, tokenize them two ways (word_tokenize vs RegexpTokenizer), remove stopwords, apply Porter and Snowball stemming and WordNet lemmatization, compare the results in a table, and track the vocabulary size at each stage.

Objective

- Understand why each preprocessing step is needed and what it changes.
- Learn how Word2Vec turns words into vectors and how to read similarity scores.
- See the difference between stemming and lemmatization on real examples.
- Understand the limits of training on a very small dataset.

2. Datasets

Both datasets were given in the assignment. They are stored as CSV files with one column called **text**.

2.1 word2vec_corpus.csv — used in Q1 (20 sentences)

Clean, grammatical sentences about diet, workouts, sleep, hydration and recovery. The two target words appear often: **diet** 8 times and **workout** 6 times (plus *workouts* 3 times).

#	Sentence
1	A balanced diet helps maintain a healthy body weight
2	Regular workout sessions improve overall muscle strength
3	Drinking enough water daily supports good digestion
4	A high protein diet is common among strength athletes
5	Cardio workouts help improve heart and lung health
6	Skipping meals can slow down your metabolism over time
7	Stretching before a workout reduces the risk of injury
8	A balanced diet should include fruits, vegetables and whole grains
9	Strength training workouts build muscle and improve bone density
10	Getting enough sleep is important for muscle recovery
11	A sudden change in diet can cause digestive discomfort
12	Yoga is a low-impact workout that improves flexibility
13	Doctors recommend a balanced diet to prevent lifestyle diseases

#	Sentence
14	Interval training workouts burn calories quickly
15	A high-sugar diet is linked to several health problems
16	Consistent workout routines improve long-term fitness levels
17	Nutritionists often design a diet plan based on individual goals
18	Warm-up exercises prepare the body for an intense workout
19	A vegetarian diet can still provide sufficient protein
20	Rest days are an important part of any workout schedule

2.2 dataset.csv — used in Q2 (12 sentences)

The same topic but deliberately messy: CAPITAL letters, repeated punctuation (!, ...), numbers (8, 7-8), symbols (&) and emoticons (:)). This is what real text from social media or reviews looks like.

#	Raw sentence
1	A balanced DIET helps maintain healthy body weight!!
2	regular workout sessions improve muscle strength... love it :)
3	drinking 8 glasses of water daily supports digestion!!
4	skipping meals can SLOW down your metabolism...
5	stretching before a workout reduces injury risk!! important
6	strength training builds muscle & improves bone density
7	getting 7-8 hrs of sleep is important for recovery :)
8	yoga is a LOW-impact workout, improves flexibility!!
9	a high-sugar diet is linked to health problems...
10	interval training burns calories QUICKLY!! great results
11	nutritionists design diet plans based on individual goals
12	rest days are an important part of any workout schedule!!

Types of noise in dataset.csv

Noise	Example	Why it is a problem
Mixed case	DIET, SLOW, QUICKLY	'DIET' and 'diet' would be counted as two different words
Punctuation	!!, ..., commas	creates useless tokens and sticks to words ('weight!!')
Numbers	8 glasses, 7-8 hrs	not useful for meaning in this task
Symbols / emoticons	&, :)	noise for a word-level analysis
Hyphenated words	low-impact, high-sugar	need to be split into separate words
Extra whitespace	left after removing symbols	creates empty tokens

3. Tools and libraries

Library	Used for
Python 3.12	programming language

Library	Used for
NLTK	word_tokenize, RegexpTokenizer, stopwords list, PorterStemmer, SnowballStemmer, WordNetLemmatizer, pos_tag
Gensim	training the Word2Vec model
pandas	loading the CSVs and showing results as tables
re, string	regular expressions and punctuation list for cleaning
Streamlit, Plotly, scikit-learn (PCA)	only for the interactive web app (extra, not required by the assignment)

4. Q1 — Word2Vec: method

What is Word2Vec?

Word2Vec is a small neural network that learns a vector (a list of numbers) for every word. It is trained on a simple task: given a word, predict the words around it (skip-gram), or given the surrounding words, predict the middle word (CBOW). Words that appear in similar contexts end up with similar vectors. Similarity between two words is measured with **cosine similarity** (1 = same direction, 0 = unrelated).

Step 1 — Load the sentences

The 20 sentences were put into a Python list and also saved as word2vec_corpus.csv.

Step 2 and 3 — Clean and tokenize

```
def clean_sentence(text):
    text = text.lower()                # DIET -> diet
    text = text.replace("-", " ")      # low-impact -> low impact
    text = text.translate(str.maketrans("", "", string.punctuation))
    return text
```

```
cleaned = [clean_sentence(s) for s in sentences]
tokenized = [word_tokenize(s) for s in cleaned]
```

Hyphens are replaced by a space before removing punctuation; otherwise 'low-impact' would become one meaningless word 'lowimpact'.

Step 4 — Remove stopwords (optional step, done)

```
stop_words = set(stopwords.words("english"))
tokens_no_stop = [[w for w in sent if w not in stop_words] for sent in tokenized]
```

Vocabulary with stopwords: **122** words → without stopwords: **102** words. Without this step words like *a*, *is*, *the* appear in almost every sentence and become 'similar' to everything, which hides the real relationships.

Step 5 — Train the model

```
model = Word2Vec(sentences=tokens_no_stop, vector_size=50, window=3,
                  min_count=1, sg=1, epochs=500, seed=42, workers=1)
```

Parameter	Value	Reason
vector_size	50	each word becomes 50 numbers; small data does not need big vectors
window	3	3 words on each side count as context; sentences are short
min_count	1	keep every word — almost every word appears only once
sg	1 (skip-gram)	skip-gram learns better than CBOW for rare words / small data
epochs	500	many passes because there is very little data. With 200 epochs every pair of words scored ~0.99, i.e. vectors were not separated yet
seed, workers	42, 1	makes training repeatable — same output every run

5. Q1 — Results

Output	Value
Total sentences	20
Vocabulary size	102
Shape of a word vector	(50,)

Full vocabulary (most frequent first)

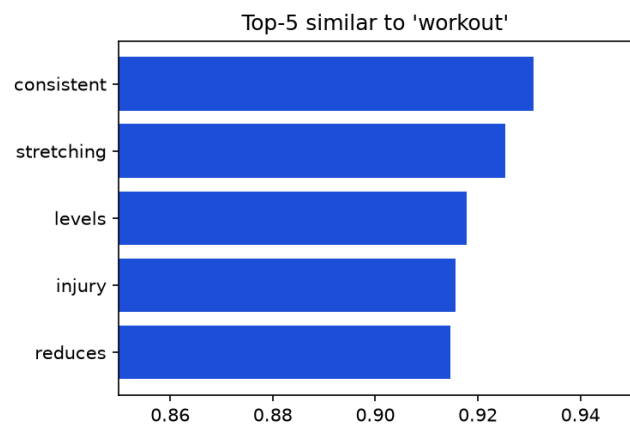
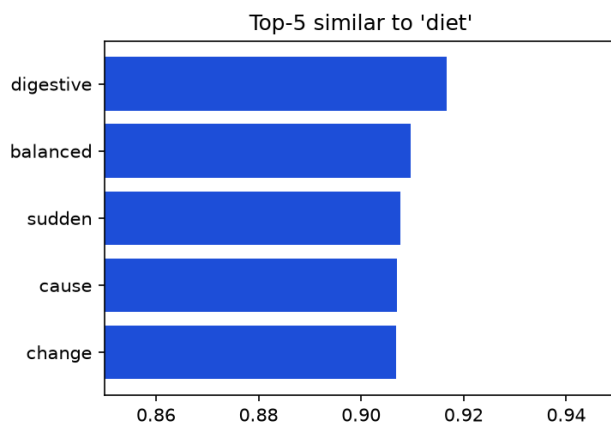
diet, workout, improve, workouts, strength, muscle, balanced, important, training, health, protein, high, enough, body, schedule, part, days, rest, sufficient, provide, still, vegetarian, intense, prepare, exercises, warm, goals, individual, based, plan, design, often, nutritionists, levels, fitness, term, long, routines, consistent, problems, several, linked, sugar, quickly, calories, burn, interval, diseases, lifestyle, prevent, recommend, doctors, flexibility, improves, impact, low, yoga, discomfort, digestive, cause, change, sudden, recovery, sleep, getting, density, bone, build, grains, whole, vegetables, fruits, include, injury, risk, reduces, stretching, time, metabolism, slow, meals, skipping, lung, heart, help, cardio, athletes, among, common, digestion, good, supports, daily, water, drinking, overall, sessions, regular, weight, healthy, maintain, helps

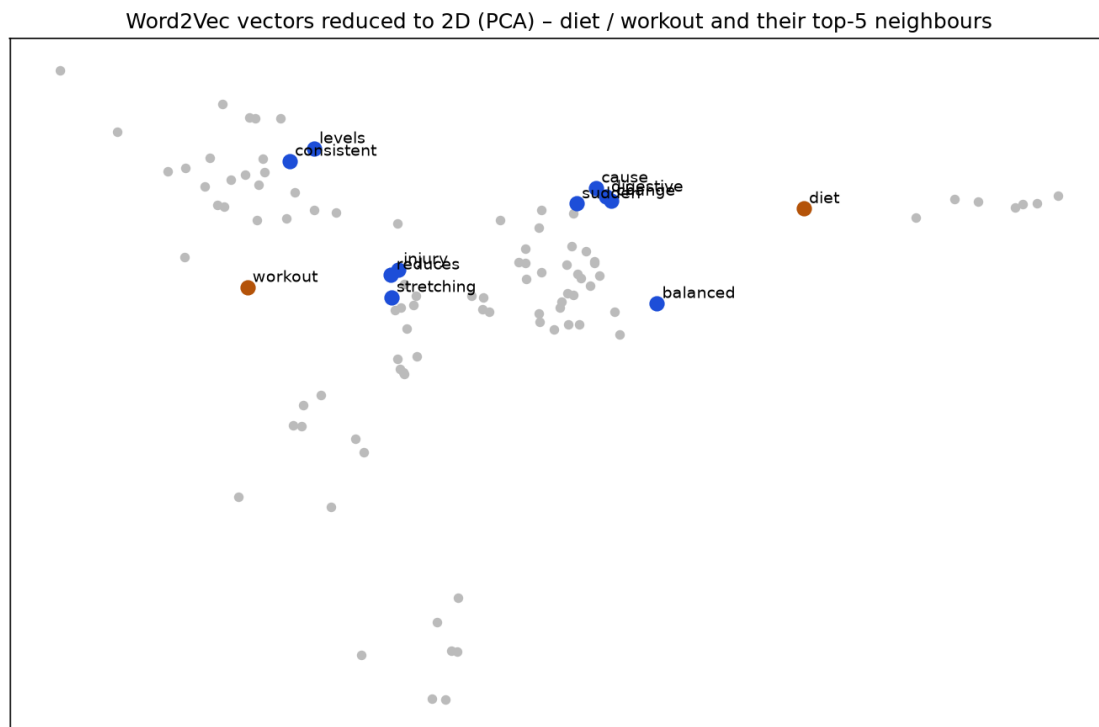
Vector for 'diet' (first 10 of 50 values)

[-0.2806 -0.1459 0.0633 0.1014 -0.078 0.1492 0.0044 0.3786 -0.1828
-0.2185]

Top-5 most similar words

Rank	Similar to 'diet'	Score	Similar to 'workout'	Score
1	digestive	0.9166	consistent	0.9308
2	balanced	0.9096	stretching	0.9253
3	sudden	0.9077	levels	0.9179
4	cause	0.9071	injury	0.9157
5	change	0.9069	reduces	0.9146





The 2D plot uses PCA to squash the 50-dimensional vectors into 2 dimensions so they can be drawn.

6. Q1 — Interpretation

Do the similar words make sense?

Partly. For **diet**, the word *balanced* makes real sense because "balanced diet" appears three times. The other neighbours — *digestive*, *sudden*, *cause*, *change* — all come from one single sentence: "a sudden change in diet can cause digestive discomfort". For **workout**, the neighbours *consistent*, *stretching*, *levels*, *injury*, *reduces* are simply words that shared a sentence with it. So the model is mostly remembering which words appeared together, not learning real meaning like $\text{diet} \approx \text{nutrition}$. With only 20 short sentences most words appear once, so one accidental co-occurrence decides a vector. On millions of sentences these accidents average out and similarities become meaningful.

Why is `min_count` needed, and what if we increase it?

`min_count` ignores words that appear fewer times than the given number. In real data this removes typos and rare words that cannot get a good vector anyway, and makes training faster. Here almost every word appears once, so increasing it destroys the vocabulary: `min_count=1` → **102** words, `2` → **14** words, `3` → **7** words.

What does increasing the window size change?

The window is how many words on each side count as context. A small window learns close, almost grammatical relationships (words directly next to each other). A large window learns broader topic relationships — words anywhere in the same sentence become similar. In our short sentences a big window basically means 'same sentence'.

Why can results differ when re-training?

Word2Vec starts from random vectors, uses random negative sampling and processes data in random order (and in parallel threads). Each run ends up slightly different, and on a tiny dataset the difference is large. Top-5 for 'diet' with different seeds:

Seed	Top-5 for 'diet'
1	protein, balanced, still, doctors, vegetarian
7	still, balanced, vegetarian, provide, sufficient
99	provide, protein, vegetarian, sudden, sufficient

Fixing seed=42 and workers=1 makes the results repeatable.

7. Q2 — Preprocessing: how the text was cleaned

The 12 raw sentences were saved to dataset.csv and loaded with pandas into a DataFrame with one column 'text'. Then a cleaning function was applied to every row and stored in a new column 'clean_text'.

```
def clean_text(text):  
    text = text.lower() # 1. lowercase  
    text = re.sub(r"[^a-z\s]", " ", text) # 2. anything not a letter/space -> space  
    text = re.sub(r"\s+", " ", text).strip() # 3. collapse extra spaces  
    return text  
  
df["clean_text"] = df["text"].apply(clean_text)
```

Step	What it does	Example
1. lower()	makes everything lowercase	'A balanced DIET' → 'a balanced diet'
2. re.sub([^\a-z\s])	replaces punctuation, digits, symbols, emoticons with a space	'7-8 hrs :)' → ' hrs '
3. re.sub(\s+)	turns many spaces into one and trims the ends	' hrs ' → 'hrs'

Why replace with a space instead of deleting? So that 'LOW-impact' becomes 'low impact' (two words) instead of 'lowimpact'.

Raw vs cleaned (side by side)

#	Raw	Cleaned
1	A balanced DIET helps maintain healthy body weight!!	a balanced diet helps maintain healthy body weight
2	regular workout sessions improve muscle strength... love it :)	regular workout sessions improve muscle strength love it
3	drinking 8 glasses of water daily supports digestion!!	drinking glasses of water daily supports digestion
4	skipping meals can SLOW down your metabolism...	skipping meals can slow down your metabolism
5	stretching before a workout reduces injury risk!! important	stretching before a workout reduces injury risk important
6	strength training builds muscle & improves bone density	strength training builds muscle improves bone density
7	getting 7-8 hrs of sleep is important for recovery :)	getting hrs of sleep is important for recovery
8	yoga is a LOW-impact workout, improves flexibility!!	yoga is a low impact workout improves flexibility
9	a high-sugar diet is linked to health problems...	a high sugar diet is linked to health problems
10	interval training burns calories QUICKLY!! great results	interval training burns calories quickly great results
11	nutritionists design diet plans based on individual goals	nutritionists design diet plans based on individual goals
12	rest days are an important part of any workout schedule!!	rest days are an important part of any workout schedule

8. Q2 — Tokenization, stopwords, stemming, lemmatization

8.1 Tokenization: word_tokenize vs RegexpTokenizer

```
regex_tok = RegexpTokenizer(r"[a-zA-Z]+") # keep only runs of letters  
word_tokenize(text) # NLTK's trained English tokenizer  
regex_tok.tokenize(text)
```

#	word_tokenize (raw text)	RegexpTokenizer (raw text)
1	A balanced DIET helps maintain healthy body weight ! !	A balanced DIET helps maintain healthy body weight
2	regular workout sessions improve muscle strength ... love it :)	regular workout sessions improve muscle strength love it
3	drinking 8 glasses of water daily supports digestion ! !	drinking glasses of water daily supports digestion

Difference: on the cleaned text both give exactly the same tokens because no punctuation is left. On the raw text, word_tokenize keeps punctuation as separate tokens ('!', '...', ':', ')') while RegexpTokenizer keeps only alphabetic words. The RegexpTokenizer output on cleaned text was used for the rest of the pipeline.

8.2 Stopword removal

Stopwords are very common words (a, is, the, of, for, it...) that carry little meaning. NLTK's English list has 198 words.

```
stop_words = set(stopwords.words("english"))
df["tokens_nostop"] = df["tokens"].apply(lambda t: [w for w in t if w not in stop_words])
```

#	Before	After	Tokens after removal
1	8	7	balanced diet helps maintain healthy body weight
2	8	7	regular workout sessions improve muscle strength love
3	7	6	drinking glasses water daily supports digestion
4	7	4	skipping meals slow metabolism
5	8	6	stretching workout reduces injury risk important
6	7	7	strength training builds muscle improves bone density
7	8	5	getting hrs sleep important recovery
8	8	6	yoga low impact workout improves flexibility
9	9	6	high sugar diet linked health problems
10	7	7	interval training burns calories quickly great results
11	8	7	nutritionists design diet plans based individual goals
12	10	6	rest days important part workout schedule

Total tokens: **95** → **74**.

8.3 Stemming (Porter and Snowball)

Stemming chops word endings using fixed rules to get a 'stem'. It is fast but the stem may not be a real word.

```
porter = PorterStemmer()
snowball = SnowballStemmer("english")
df["porter"] = df["tokens_nostop"].apply(lambda t: [porter.stem(w) for w in t])
df["snowball"] = df["tokens_nostop"].apply(lambda t: [snowball.stem(w) for w in t])
```

8.4 Lemmatization (WordNet, with POS tags)

Lemmatization looks the word up in the WordNet dictionary and returns its base form (lemma). It needs to know the part of speech; without it every word is treated as a noun and 'improves' stays 'improves'. So nltk.pos_tag was used and the tags were mapped to WordNet tags.

```
def to_wordnet_pos(tag):
    if tag.startswith("J"): return wordnet.ADJ
    if tag.startswith("V"): return wordnet.VERB
    if tag.startswith("R"): return wordnet.ADV
    return wordnet.NOUN

def lemmatize(tokens):
    return [lemmatizer.lemmatize(w, to_wordnet_pos(t)) for w, t in nltk.pos_tag(tokens)]
```

#	Porter	Snowball	Lemma
1	balanc diet help maintain healthi bodi weight	balanc diet help maintain healthi bodi weight	balance diet help maintain healthy body weight
2	regular workout session improv muscl strength love	regular workout session improv muscl strength love	regular workout session improve muscle strength love
3	drink glass water daili support digest	drink glass water daili support digest	drinking glass water daily support digestion
4	skip meal slow metabol	skip meal slow metabol	skip meal slow metabolism
5	stretch workout reduc injuri risk import	stretch workout reduc injuri risk import	stretch workout reduces injury risk important
6	strength train build muscl improv bone densiti	strength train build muscl improv bone densiti	strength training build muscle improve bone density
7	get hr sleep import recoveri	get hrs sleep import recoveri	get hrs sleep important recovery
8	yoga low impact workout improv flexibl	yoga low impact workout improv flexibl	yoga low impact workout improve flexibility
9	high sugar diet link health problem	high sugar diet link health problem	high sugar diet link health problem
10	interv train burn calori quickli great result	interv train burn calori quick great result	interval training burn calorie quickly great result
11	nutritionist design diet plan base individu goal	nutritionist design diet plan base individu goal	nutritionist design diet plan base individual goal
12	rest day import part workout schedul	rest day import part workout schedul	rest day important part workout schedule

9. Q2 — Comparison and vocabulary

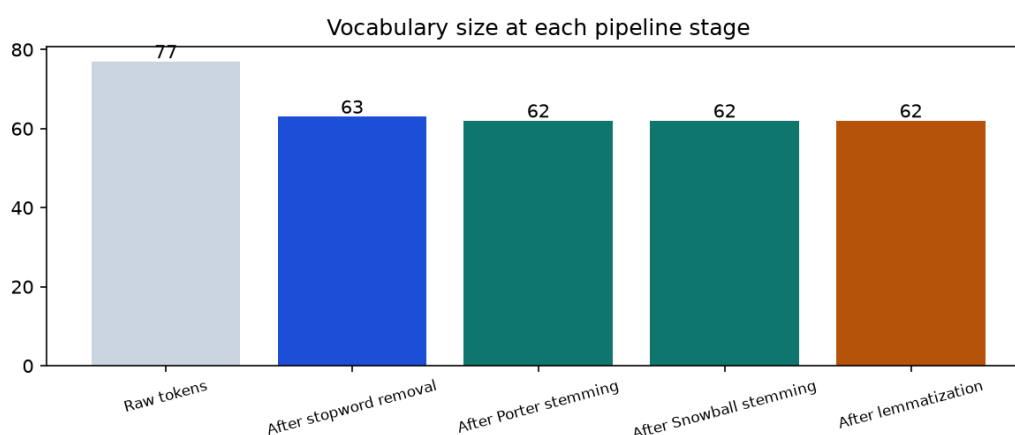
Comparison table (rows where Porter and Snowball disagree are highlighted)

Original word	Porter stem	Snowball stem	Lemma
balanced	balanc	balanc	balance
sessions	session	session	session
improves	improv	improv	improve
drinking	drink	drink	drinking
glasses	glass	glass	glass
supports	support	support	support
digestion	digest	digest	digestion
skipping	skip	skip	skip
metabolism	metabol	metabol	metabolism
stretching	stretch	stretch	stretch

Original word	Porter stem	Snowball stem	Lemma
builds	build	build	build
density	densiti	densiti	density
calories	calori	calori	calorie
quickly	quickli	quick	quickly
hrs	hr	hrs	hrs
nutritionists	nutritionist	nutritionist	nutritionist
plans	plan	plan	plan
days	day	day	day

Vocabulary size at each stage

Stage	Vocabulary size
Raw tokens	77
After stopword removal	63
After Porter stemming	62
After Snowball stemming	62
After lemmatization	62



Stopword removal gives the biggest drop. Stemming and lemmatization only merge a few words here (e.g. *improve* / *improves*) because the dataset is small and most words appear in one form only.

10. Q2 — Interpretation

Stemming vs lemmatization on this dataset

Both reduce the vocabulary by merging forms of the same word. Stemming cuts endings by rules, so it is fast but produces non-words like *balanc*, *calori*, *densiti*, *metabol*. Lemmatization uses the dictionary plus POS tags and returns real words like *improve*, *build*, *burn*, but it is slower and depends on correct POS tags. For a **search engine** I would use stemming — speed and matching more variants (better recall) matter more than readability, and users never see the stems. For **classification** or anything people read (topic labels, word clouds), I would use lemmatization because the tokens stay meaningful.

Why can stemming produce non-dictionary words while lemmatization always returns a valid word?

A stemmer is just a list of suffix rules (remove '-ing', turn '-ies' into '-i', ...). It never checks whether the result is a real word, so 'calories' → 'calori' and 'density' → 'densiti'. A lemmatizer looks the word up in WordNet and returns the dictionary base form, so the output is always a real word.

Practical difference between RegexpTokenizer and word_tokenize?

word_tokenize is a trained English tokenizer: it splits punctuation into separate tokens and handles contractions ("don't" → 'do', "n't"). RegexpTokenizer only returns what matches the pattern, so with [a-zA-Z]+ it keeps letters only. Use word_tokenize when punctuation matters (sentiment — '!!' and ':') carry emotion — or POS tagging); use RegexpTokenizer when you want clean words fast and full control over what a token is.

A word where Porter and Snowball differ, and why

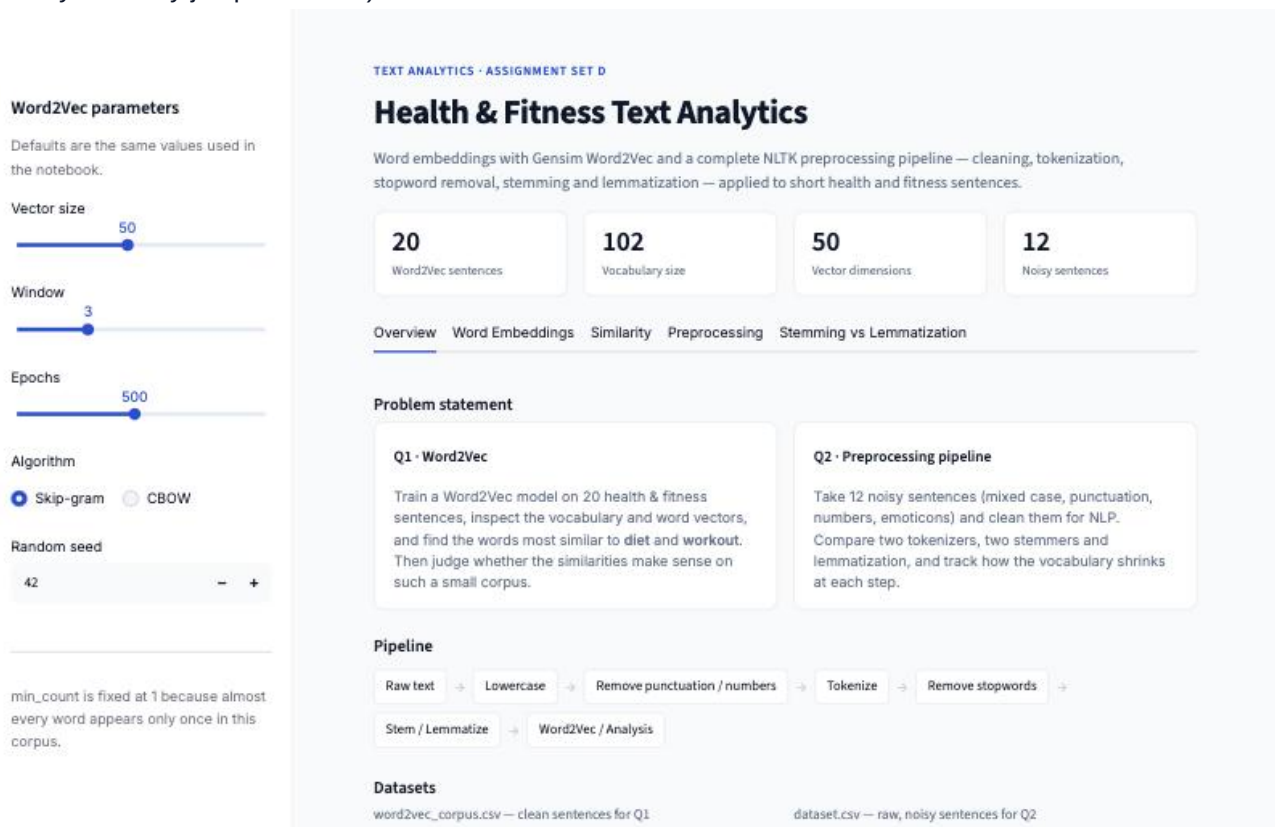
quickly → Porter 'quickli', Snowball 'quick'; **hrs** → Porter 'hr', Snowball 'hrs'. Snowball (also called Porter2) is Martin Porter's improved version of his own algorithm. It has a special rule for '-ly' adverbs, handles the final 'y' differently, and leaves very short words alone. Because the rules are different the stems can differ. Outside the dataset: 'generously' → gener vs generous, 'fairly' → fairli vs fair.

11. Interactive web app (extra)

Besides the notebook, the project has a small Streamlit web app (app.py) that runs the same code and lets you explore the results interactively. It is not required by the assignment; it is there to demonstrate the work.

Tab	What it shows
Overview	problem statement, pipeline diagram, both datasets
Word Embeddings	all words plotted in 2D with PCA; choose a word to highlight its top-5 neighbours; full vocabulary
Similarity	top-5 similar words for diet and workout as bar charts, the raw vector, simple vector arithmetic
Preprocessing	type any sentence and watch each step: cleaning, both tokenizers, stopwords, stemming, lemmatization; vocabulary chart
Stemming vs Lemmatization	Porter / Snowball / Lemma for every word, disagreements highlighted; test any word

The sidebar has sliders for vector_size, window, epochs, algorithm and seed — the model retraines instantly, which makes it easy to show what each parameter does (for example, set epochs to 100 and every similarity jumps to ~0.99).



12. Project structure and how to run

```
ml_bba/
|-- TextAnalytics_SetD.ipynb      notebook: Q1 + Q2 with all outputs
|-- data/
|   |-- word2vec_corpus.csv       Q1 corpus (20 sentences)
|   |-- dataset.csv              Q2 raw dataset (12 sentences)
|-- docs/
|   |-- Project_Explanation.docx  submission write-up
|   |-- Project_Report.pdf        this report
```

```

|-- app.py                      Streamlit app
|-- requirements.txt
`-- README.md

pip install -r requirements.txt
streamlit run app.py           # web app
jupyter notebook               # then open TextAnalytics_SetD.ipynb

```

13. Quick revision: likely viva questions

Question	Answer
What is NLP preprocessing and why is it needed?	Raw text is noisy. Preprocessing standardises it (case, punctuation, word forms) so the same word is always counted as the same token and the model focuses on meaning.
What is a token?	A single unit of text after splitting — usually a word.
What are stopwords?	Very frequent words like 'the', 'is', 'a' that carry little meaning. Removing them reduces noise and size.
Stemming vs lemmatization in one line?	Stemming cuts endings by rules (fast, may give non-words); lemmatization uses a dictionary (slower, always real words).
Why use POS tags for lemmatization?	The lemma depends on the part of speech: 'builds' as a verb → 'build'. Without a tag WordNet assumes noun.
What is Word2Vec?	A shallow neural network that learns a dense vector for each word from the words around it.
Skip-gram vs CBOW?	Skip-gram predicts context words from the centre word (better for rare words, small data). CBOW predicts the centre word from context (faster).
What is cosine similarity?	The cosine of the angle between two vectors: 1 = same direction, 0 = unrelated, -1 = opposite.
What is vector_size?	How many numbers represent each word (here 50).
Why are the results noisy?	Only 20 sentences; most words appear once, so the model learns sentence co-occurrence, not real meaning.
Why replace hyphens with spaces?	So 'low-impact' becomes 'low' and 'impact', not the fake word 'lowimpact'.
Why did vocabulary drop most at stopword removal?	Stopwords are repeated in almost every sentence; stemming/lemmatization only merge a few words in such a small dataset.
What is PCA in the app?	Principal Component Analysis — reduces 50-dimensional vectors to 2 so they can be plotted.